“Diary Application with Autocomplete”

Overview:

This program is a **console-based diary application** which allows users to:

1) Type diary entries (with real-time word suggestions via autocomplete)
2) Auto-save unknown words to a dictionary file
3) Save diary entries with the current date
4) View all entries
5) Search diary entries by date

Data Structures Used:

1.*Trie (Prefix Tree)*

Purpose:

Efficiently store words for quick prefix-based suggestions.

Structure:

```
// Trie Node
struct TrieNode {
    bool isEndOfWord;
    unordered_map<char, TrieNode*> children;
    TrieNode() {
        isEndOfWord = false;
    }
};
```

Features:

- ✓ Each character is a node.
- ✓ Children stored in an *unordered_map<char, TrieNode*>.*
- ✓ *isEndOfWord* marks the end of a valid word.
- ✓ Supports insertions and prefix suggestions.

2. Vector (wordList)

Purpose:

Maintain a list of all known dictionary words loaded from file.

Structure:

```
// Global word list and diary head
vector<string> wordList;
DiaryEntry* diaryHead = nullptr;
```

Features:

- ✓ Prevents duplicate word insertions.
- ✓ Used for quick lookup before adding new words.

3. Singly Linked List (DiaryEntry)

Purpose:

Store diary entries dynamically.

Structure:

```
struct DiaryEntry {
    string date;
    string content;
    DiaryEntry* next;
    DiaryEntry(string d, string c) {
        date = d;
        content = c;
        next = nullptr;
    }
};
```

Features:

- ✓ Each entry has a date, content, and a pointer to the next entry.
- ✓ Enables easy addition of new entries at the end.
- ✓ Supports traversing through all entries.
- ✓ Search entries by date.

Program Flow:

1. Program Initialization

- ✓ Load words from dictionary.txt into wordList and insert them into the trie.
- ✓ Get current date using: *string getCurrentDate()*
- ✓ Start capturing diary input character by character.

2.Diary Entry Input (with Autocomplete)

While typing:

- ✓ Each character added to *currentWord*.
- ✓ On pressing **space** or **Enter**:
- ✓ Autocomplete suggestions are displayed using trie.displaySuggestions().
- ✓ If no suggestion → add word to dictionary and trie.
- ✓ Append word to sentence.
- ✓ Reset currentWord.

Double Enter: saves the current entry and exits typing mode.

```
Diary initialized for: 2025-06-01
Start typing your diary entry (ESC = end typing, Double Enter = save and exit):
Sentence so far:
hey there
what's up? long t
Suggestions for 't':
the
there!
that
that's
te
today.
Total suggestions: 6
```

After Typing

- ✓ Save the diary entry : *addDiaryEntry(currentDate, fullSentence)*;
- ✓ Show main diary menu:

```
Diary Menu
1. View All Entries
2. Find Entry by Date
3. Exit
Enter choice:
```

Key Features:

Feature	Description
Trie Autocomplete	Real-time word suggestions while typing diary entries.
Auto Word Save	New unknown words automatically added to the dictionary and trie.
Date Initialization	Diary initialized with the current system date.
Linked List Diary Storage	Entries stored as linked list nodes with date and content.
View All Entries	Displays all stored diary entries.
Search by Date	Allows searching for specific diary entries by date.

Advantages:

- ✅ Fast prefix-based word suggestion using trie.
- ✅ Dynamic storage of diary entries via linked list.
- ✅ Persistent word dictionary via file storage.
- ✅ Simple, clean user interaction in console.

Diagram — Example Trie Structure for Words

["help", "hello", "hell"]

```
root
└── 'h'
    └── 'e'
        └── 'l'
            ├── 'l' (isEndOfWord)
            │   └── 'o' (isEndOfWord)
            └── 'p' (isEndOfWord)
```

Time Complexity(Runtime)

Operation	Time Complexity (Big O)	Explanation
Insert word into Trie	O(L)	L is length of the word. Each character is inserted sequentially into trie nodes.
Display suggestions (Autocomplete)	O(P + S × A)	P is length of prefix. S is number of suggestions, A is average word length. Collect words using DFS.
Load words from file into Trie	O(N × L)	N is number of words in file, L average word length. Each word is inserted one by one.
Add word to wordList (vector)	O(1) (amortized) for push_back, O(N) for find()	push_back is O(1), but find to check for duplicates is linear.
Add diary entry (linked list)	O(E)	E is number of diary entries (traversing to end of list for insertion).
View all diary entries	O(E)	Traversing linked list to print entries.
Find entry by date	O(E × D)	E is number of entries, D is average length of date strings (constant in this case).
Get current date	O(1)	Uses system time, constant time operation.
Typing loop per character (display suggestions)	O(P + S × A) per word (see above)	Because for each completed word (on space/Enter) it calls displaySuggestions.

Space Complexity (Memory)

Data Structure	Space Complexity	Explanation
Trie	O(T × L)	T is total number of unique characters inserted, L is average word length. Each node has a map of possible next characters.
wordList (vector)	O(N × L)	N is number of words, L is average length.
Diary entries (linked list)	O(E × C)	E is number of entries, C is average content length.
Suggestions (vector inside displaySuggestions)	O(S × A)	S is number of suggestions, A is average word length. Temporarily created per suggestion call.

Total (Approximate Combined)

Worst-case Time per Entry: $O(E + P + S \times A)$

E for diary traversal

$P + S \times A$ for autocomplete suggestions

Worst-case Space (Memory Used Together):

$O(T \times L + N \times L + E \times C)$

Conclusion:

This program is a **simple yet functional diary application** integrating:

Data structures like trie, vector, linked list.

File handling for persistent dictionary.

Dynamic input processing with real-time suggestions

Date-wise diary management

It's an excellent practical example of applying data structures to a real-world inspired application.

“Diary Application with Autocomplete”

Project and Report by: MUSTABSHIRAH AHMED
Roll.No: (B23110006127)

Data Structure And Algorithm

Course Incharge: Ms. Maryam Feroze